



Compiler Design Lab, Course Code - CSE314

PROJECT PROPOSAL

OF

**COMPILER VISUALIZER SWING – STEPWISE LEARNING TOOL FOR
COMPILER PHASES**

65-H2 , GROUP- 05	
NAME	ID
Shafin Ahmed	232-15-184
MD FOYSAL BHUIYAN	232-15-897
SAFAYET ABIR	232-15-225
ABU DAZANA	232-15-810
Bijoy krishna Sarker	232-15-219

This Project Proposal is Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering

Supervised By

.....

Rabeya Khatun

Lecturer

Department of CSE

DAFFODIL INTERNATIONAL UNIVERSITY

DHAKA, BANGLADESH

Nov 25, 2025

Project Title:

Compiler Visualizer Swing – Stepwise Learning Tool for Compiler Phases

1. Project Overview

Summary:

We are planning to develop a **Java Swing-based Compiler Visualizer** that helps students and beginners understand how a compiler works. This project allows users to input expressions and see how the compiler processes them step by step. Each compiler phase, from **Lexical Analysis to Assembly Code Generation**, will be displayed in separate tabs with visual elements. Our goal is to make learning compiler design **interactive, visual, and easier to grasp**.

Background:

Learning compiler design is often difficult for students because the concepts are abstract and theoretical. Usually, textbooks explain how compilers work, but it is hard to see the practical steps behind tokenization, parsing, and code generation. This project will provide a **hands-on and visual learning experience**. Users will see how expressions are tokenized, parsed, checked semantically, converted to intermediate code, optimized, and finally translated into assembly. By creating this tool, we aim to make compiler learning more **engaging and interactive**.

2. Objectives

- To build a **user-friendly, modular, and visual tool** in Java Swing.
- To implement **Lexical Analysis** including tokens, symbol table, and normalized expressions.
- To create **Syntax Analysis** visualization using bubble-style parse trees.
- To implement **Semantic Analysis, Intermediate Code Generation, Optimization, and Assembly Generation** modules.
- To allow a single **Input Expression tab** to propagate expressions to all compiler phase tabs automatically.
- To include **educational theory sections** in each tab, so users can understand the concepts along with the visuals.
- To develop a tool that can be **used in university courses** for learning compiler design in a practical way.

3. Research Questions / Hypotheses

- This project explores how visualizing compiler phases can improve understanding among students by **presenting the entire process** in a more interactive and intuitive way. It also examines whether automatic token generation, symbol table creation, and parse tree visualization can make the learning experience smoother and more effective.
- We assume that a **step-by-step**, interactive compiler visualizer will help students understand compiler concepts more quickly and clearly compared to studying the material through textbooks alone.

4. Methodology

Materials and Equipment:

- Java Development Kit (JDK 17 or above)
- IntelliJ IDEA or Eclipse IDE
- Java Swing for GUI development
- Standard computer systems for development and testing

Experimental Design:

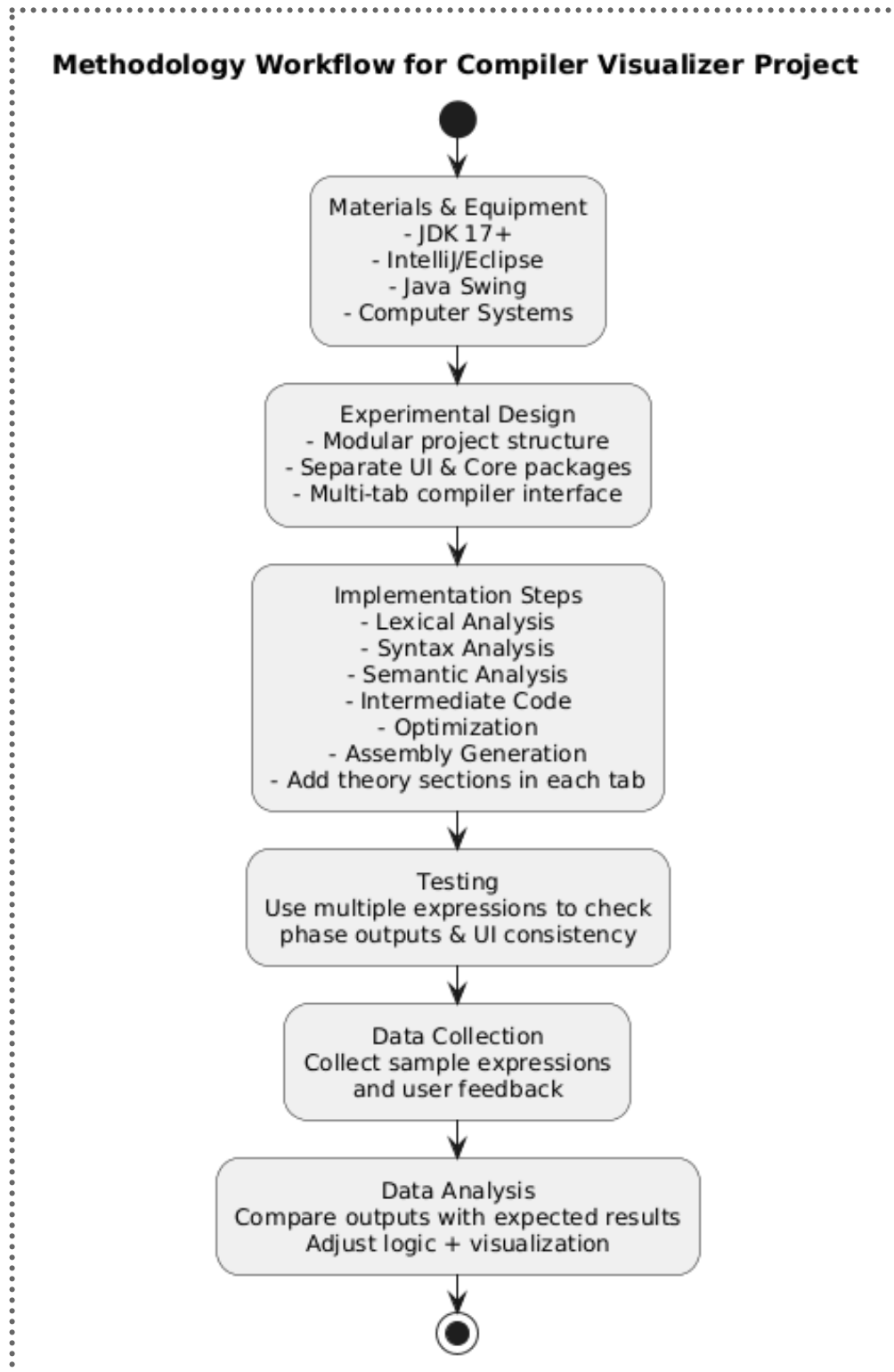
1. Design a **modular project structure**, separating GUI (ui package) and core logic (core package).
2. Create each compiler phase as a **separate tab in the GUI**, connected to the Input Expression tab.
3. Implement **Lexical Analysis** to generate tokens, symbol tables, and normalized expressions.
4. Implement **Syntax Analysis** with a bubble-style parse tree visualization.
5. Develop **Semantic Analysis, Intermediate Code, Optimization, and Assembly** modules.
6. Include **theory sections** in each tab to explain the concepts clearly.
7. Test with multiple expressions to ensure **correct output and smooth interaction** between tabs.

Data Collection:

- We will collect sample expressions from test users and analyze the outputs from each compiler phase.
- User feedback will be recorded to evaluate usability and clarity.

Data Analysis:

- Compare generated tokens, symbol tables, parse trees, and intermediate code with expected results.
- Adjust logic and GUI visualization based on test feedback.



5. Compiler Workflow Overview Diagram

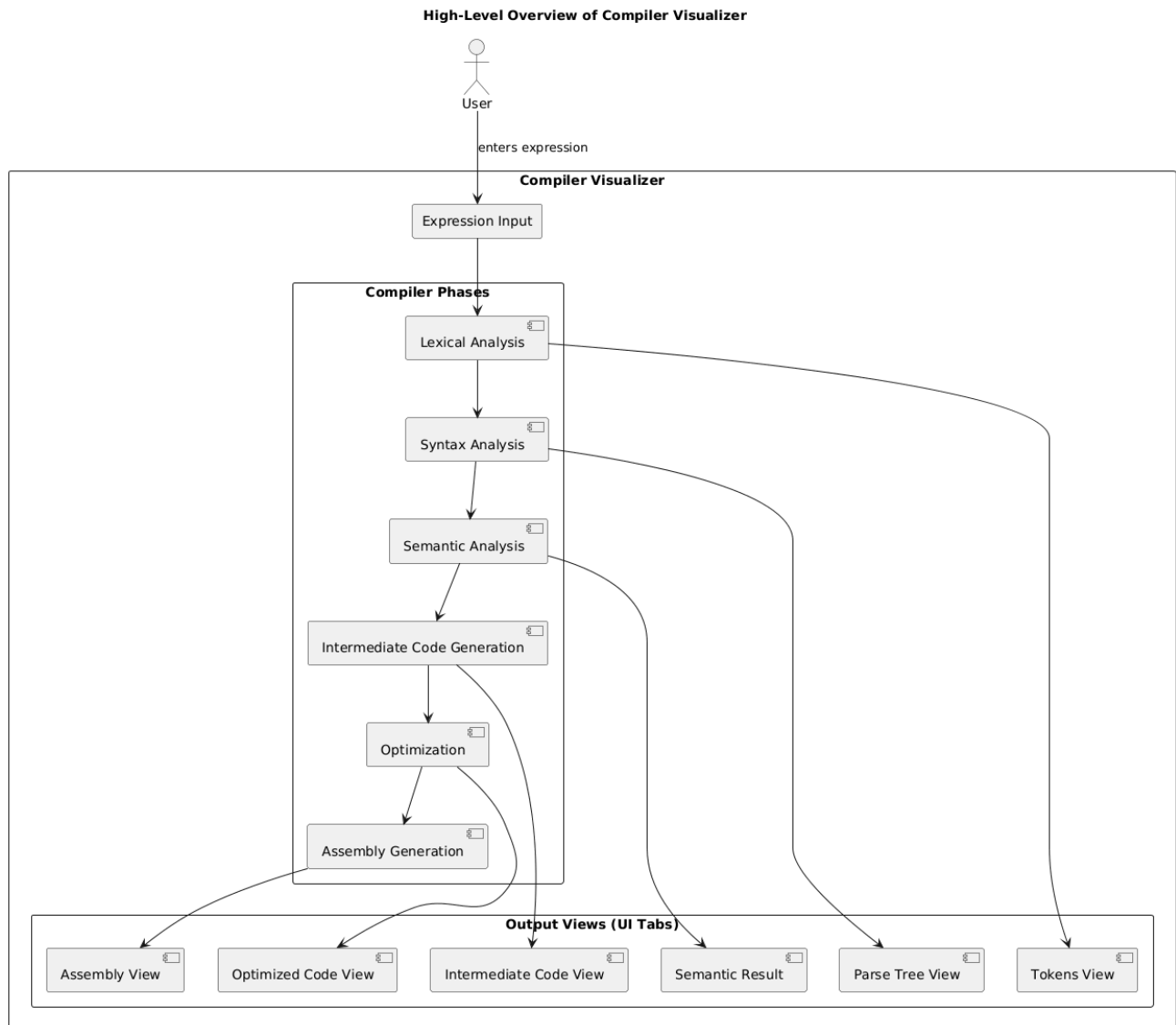


Figure 5.1: Project Workflow Overview

This diagram presents the overall flow of our compiler visualizer project.

It shows how a user's expression moves step-by-step through each compiler phase, starting from lexical analysis and ending in assembly generation.

The purpose of this visual is to give a clear early-stage understanding of how all modules are connected and how data flows through the system.

Including this diagram in the proposal helps explain the project structure even before full development begins.

6. Project Timeline

Phase	Description	Expected Completion Date
Planning	Design project structure, decide package layout, and plan GUI	11-Nov-2025
Implementation	Build GUI tabs, integrate core compiler modules	18-Nov-2025
Testing & Refinement	Test the compiler phases with multiple expressions, debug and fix issues	09-Dec-2026
Reporting	Prepare documentation, screenshots, and final report	15-Dec-2026

7. Potential Challenges and Risks

- **Complexity in parse tree visualization** for complex expressions.
- Ensuring **accurate symbol table creation** and token mapping.
- Synchronizing the **Input Expression tab** with all compiler phase tabs.
- GUI responsiveness when handling large expressions.
- Risk mitigation: we will test each module individually, start with simple expressions, and gradually increase complexity.

8. Expected Outcomes and Impact

- A **fully functional Swing-based compiler visualizer** showing all six phases of compilation.
- Interactive **visual representation of compiler phases**, making it easier for beginners to understand.
- Detailed **symbol tables, tokens, normalized expressions, parse trees, and theory sections** for each phase.
- This tool will help students learn compiler design in a **practical and engaging way**.
- The project could be extended for **web-based or AI-assisted compiler visualization** in the future.

9. References

- Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D. **Compilers: Principles, Techniques, and Tools**. Pearson, 2006.

- Grüne, D., van Reeuwijk, K. **Modern Compiler Design**. Springer, 2012.
- Appel, A.W. **Modern Compiler Implementation in Java**. Cambridge University Press, 2002.
- Oracle Java Documentation: <https://docs.oracle.com/en/java/javase/>

10. Project Team and Responsibilities

Team Member	Role	Responsibilities
Shafin Ahmed (232-15-184)	Project Lead / UI Lead	Oversee project, design MainApp, PhaseTabs, InputExpressionTab, and overall GUI, Assists to build all core logics
MD FOYSAL BHUIYAN (232-15-897)	Core Logic Developer	Main core logic developer : Implement LexicalAnalyzer, SyntaxAnalyzer, SemanticAnalyzer modules
SAFAYET ABIR (232-15-225)	GUI Developer	Develop Swing-based tabs, integrate visuals and interaction, also assists to develop 2-3 classes
ABU DAZANA (232-15-810)	Documentation Lead	Write theory content, maintain reports, screenshots, and final proposal also assists to develop 2-3 classes
Bijoy krishna Sarker (232-15-219)	Testing & QA	Test all modules, debug, collect user feedback, and ensure smooth functionality, also assists to develop 2-3 classes

11. Approval

Project Supervisor Signature: _____

Date of Approval: _____